Proyecto Integrador

Snake Game

Python

PRESENTADO POR

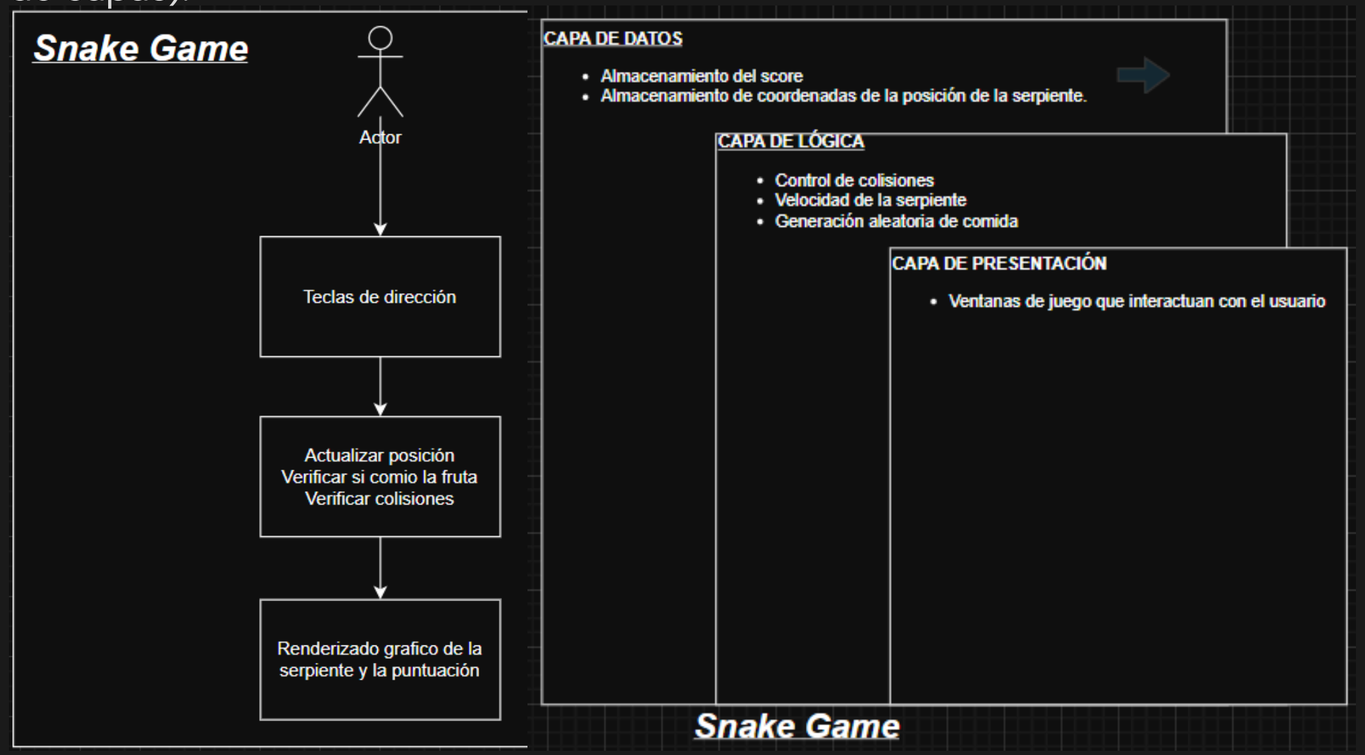
CRISTIAN ALEXANDER VINOCUNGA LEMA

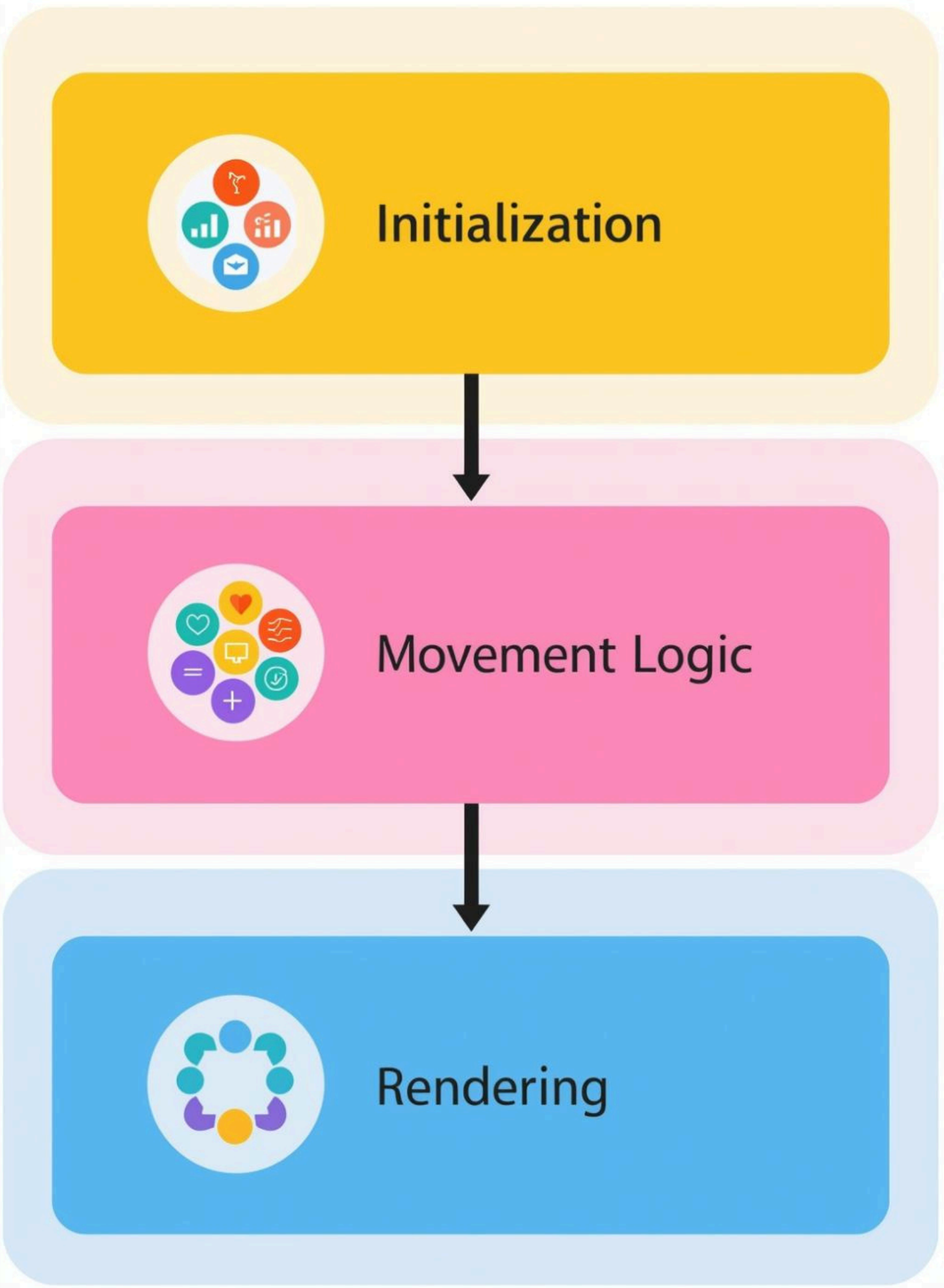
Introducción y Objetivos

Este proyecto tiene como objetivo principal desarrollar el “Clásico juego de la Serpiente” que integre **lógica imperativa y funcional**, abordando la gestión de estados de juego y eventos de usuario en tiempo real.

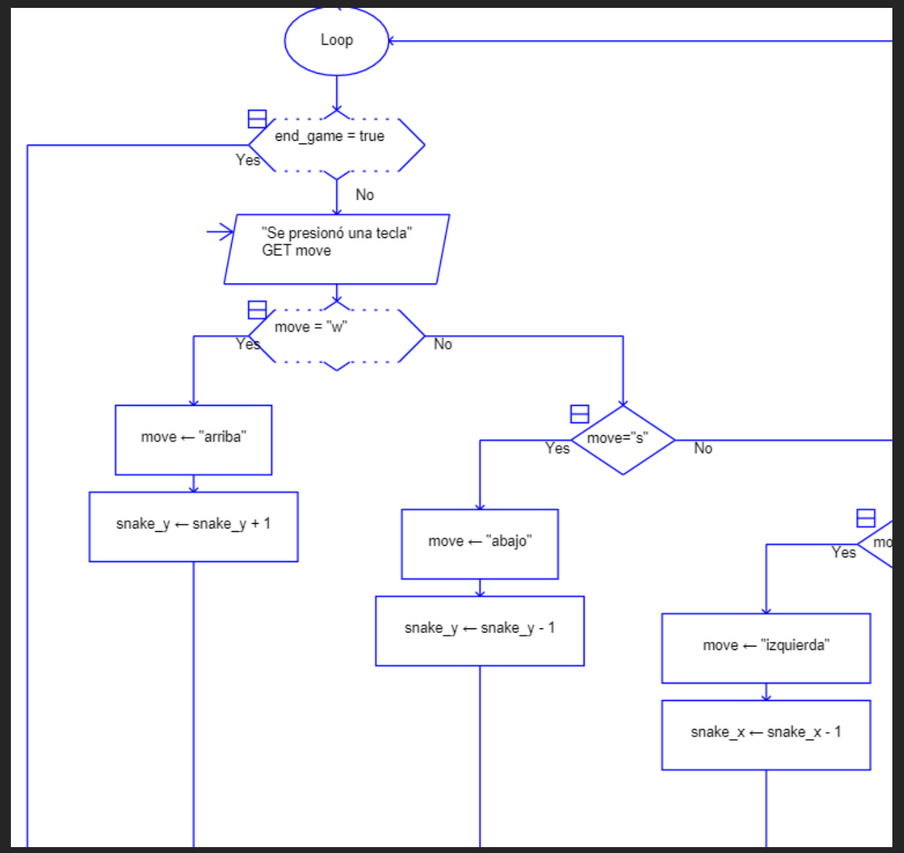
Arquitectura del Sistema

En esta sección, exploramos la **arquitectura** del juego "Snake Game", representados por medio de un diagrama de funcionalidad (Casos de Uso UML) y un diagrama de arquitectura (Arquitectura de capas).





Lógica Estructural



GAME LOOP

El **Game Loop** es fundamental para la ejecución del juego, utilizando while True para mantener el ciclo activo, gestionando el estado y la lógica en tiempo real.

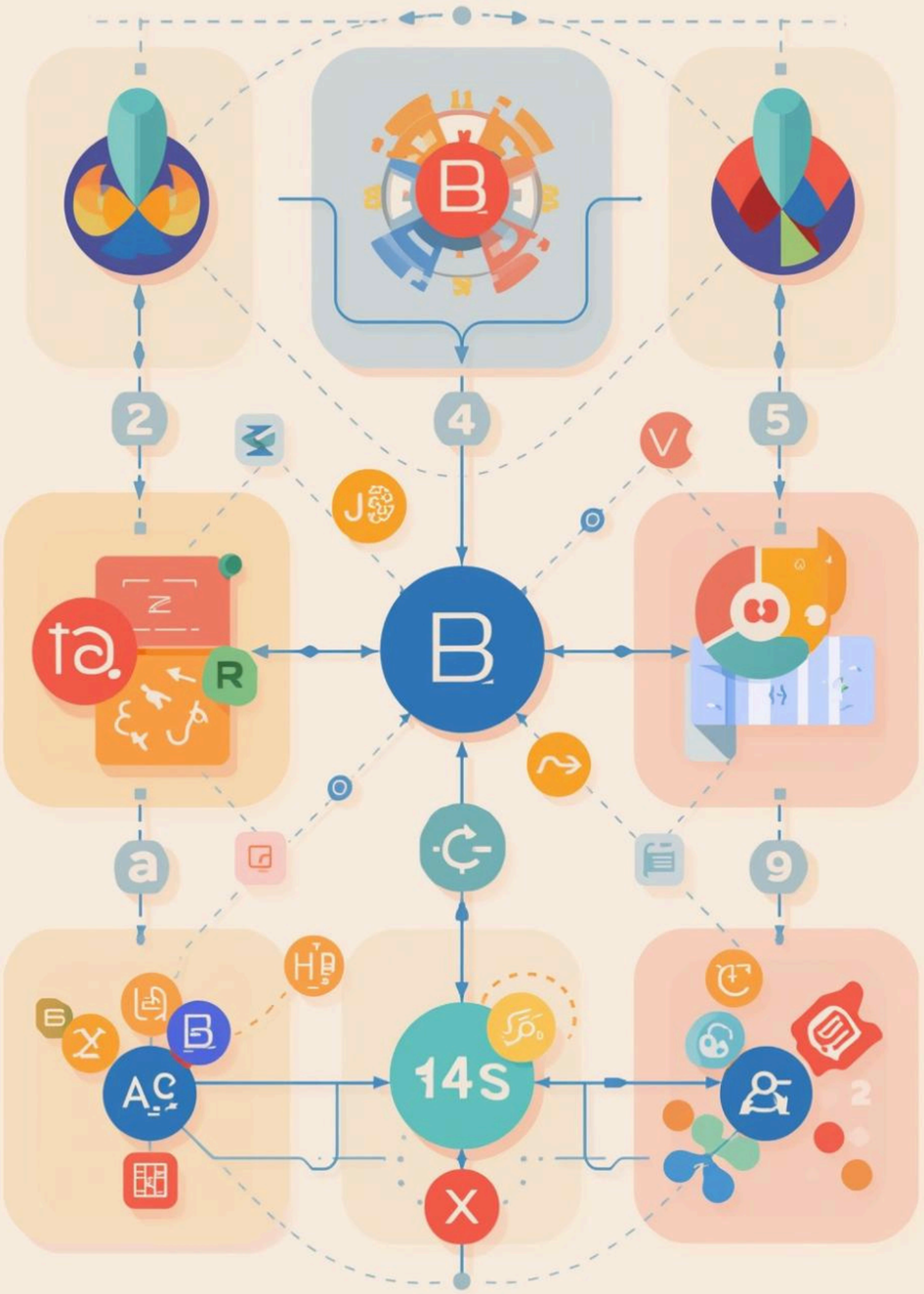
CRECIMIENTO DINÁMICO

La **serpiente** crece dinámicamente mediante listas que almacenan las posiciones de cada segmento, permitiendo una expansión fluida conforme el jugador consume alimento y el juego avanza.

Programación Funcional

En esta sección, exploramos cómo la **programación funcional** se aplica en el desarrollo del juego, usando funciones para mejorar la modularidad y la claridad del código.

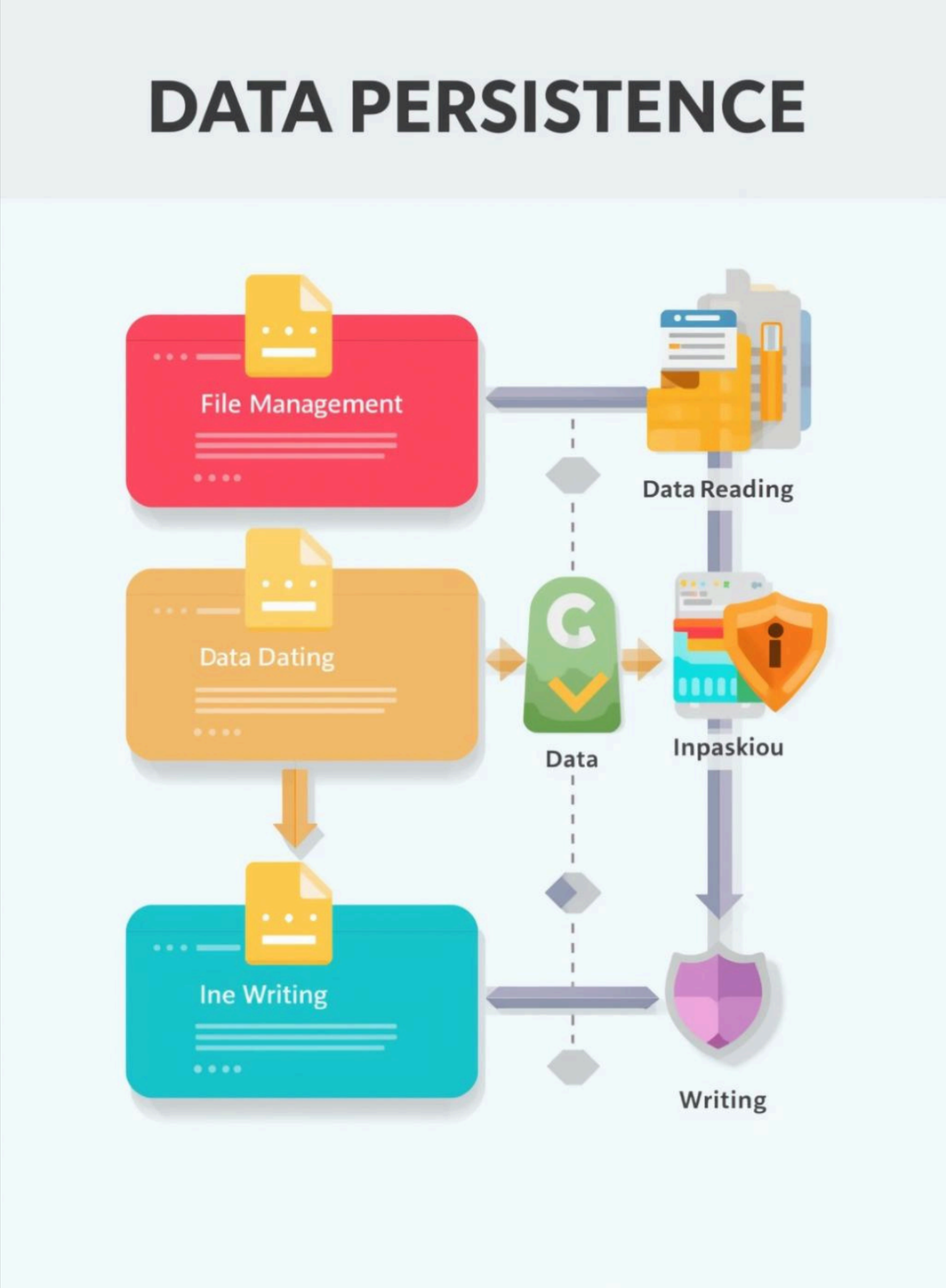
```python
# Colisión con bordes (Usando la lógica de la Unidad 4)
if esta_fuera_limites(cabeza.xcor(), 280) or esta_fuera_limites(cabeza.ycor(), 280):
    time.sleep(1)
    cabeza.goto(0,0)
    cabeza.direction = "stop"

    # Esconder segmentos (Reiniciar juego)
    for s in segmentos:
        s.goto(1000, 1000)
    segmentos.clear()

    marcador = 0
    texto.clear()
    texto.write(f"Puntos: {marcador}  Récord: {max_puntaje}", align="center", font=("Courier", 24, "normal"))

# Colisión con comida
if calcular_distancia(cabeza, comida) < 20:
    comida.goto(random.randint(-280, 280), random.randint(-280, 280))

    # Nuevo segmento (Estructura de datos - Unidad 3)
    nuevo_segmento = turtle.Turtle()
    nuevo_segmento.speed(0)
    nuevo_segmento.shape("square")
    nuevo_segmento.color("grey")
    nuevo_segmento.penup()
    segmentos.append(nuevo_segmento)

    marcador += 1
    if marcador > max_puntaje:
        max_puntaje = marcador
        guardar_max_puntaje(max_puntaje) # Persistencia - Unidad 4

    texto.clear()
    texto.write(f"Puntos: {marcador}  Récord: {max_puntaje}", align="center", font=("Courier", 24, "normal"))

# Mover el cuerpo (Lógica de seguimiento)
for i in range(len(segmentos) - 1, 0, -1):
    x = segmentos[i-1].xcor()
    y = segmentos[i-1].ycor()
    segmentos[i].goto(x, y)
```

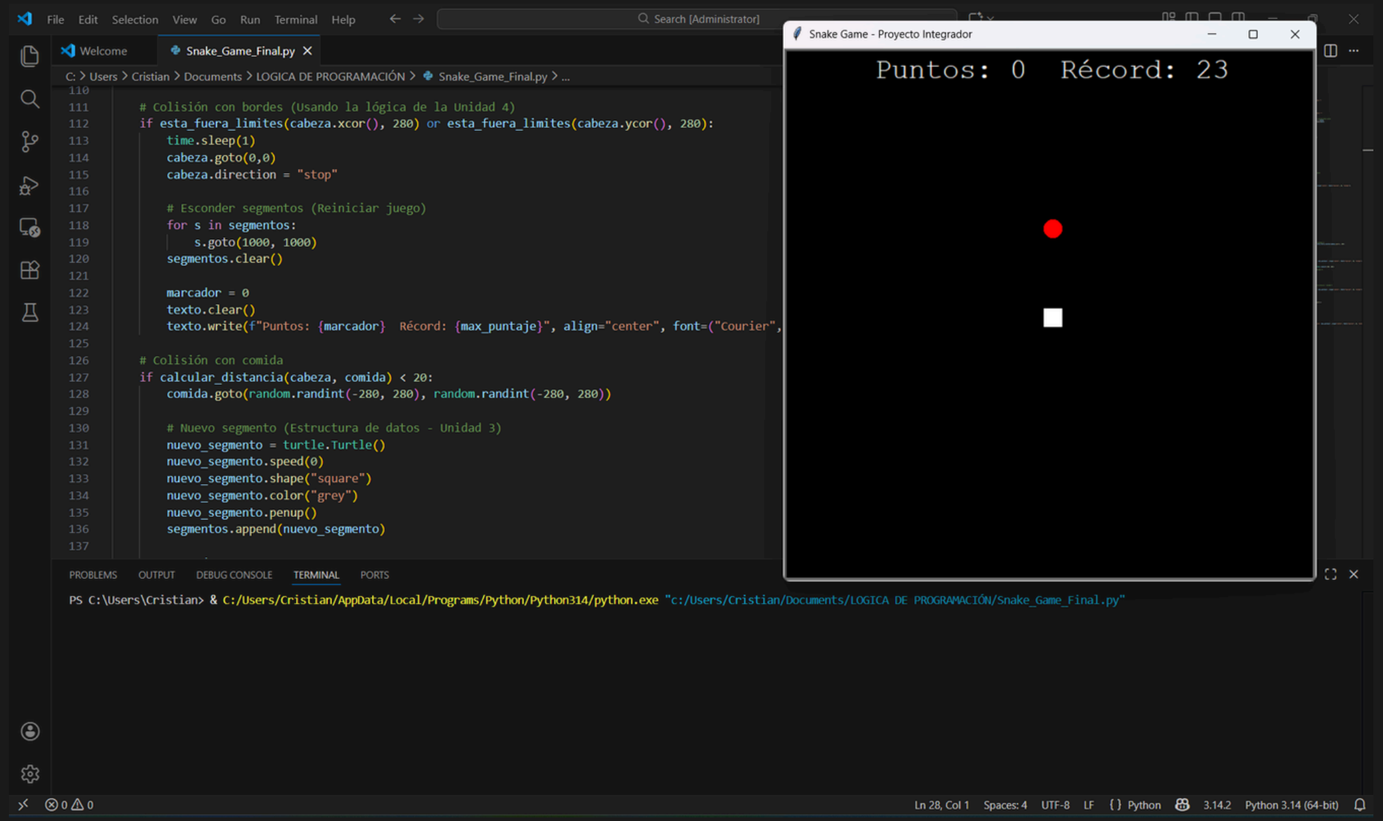
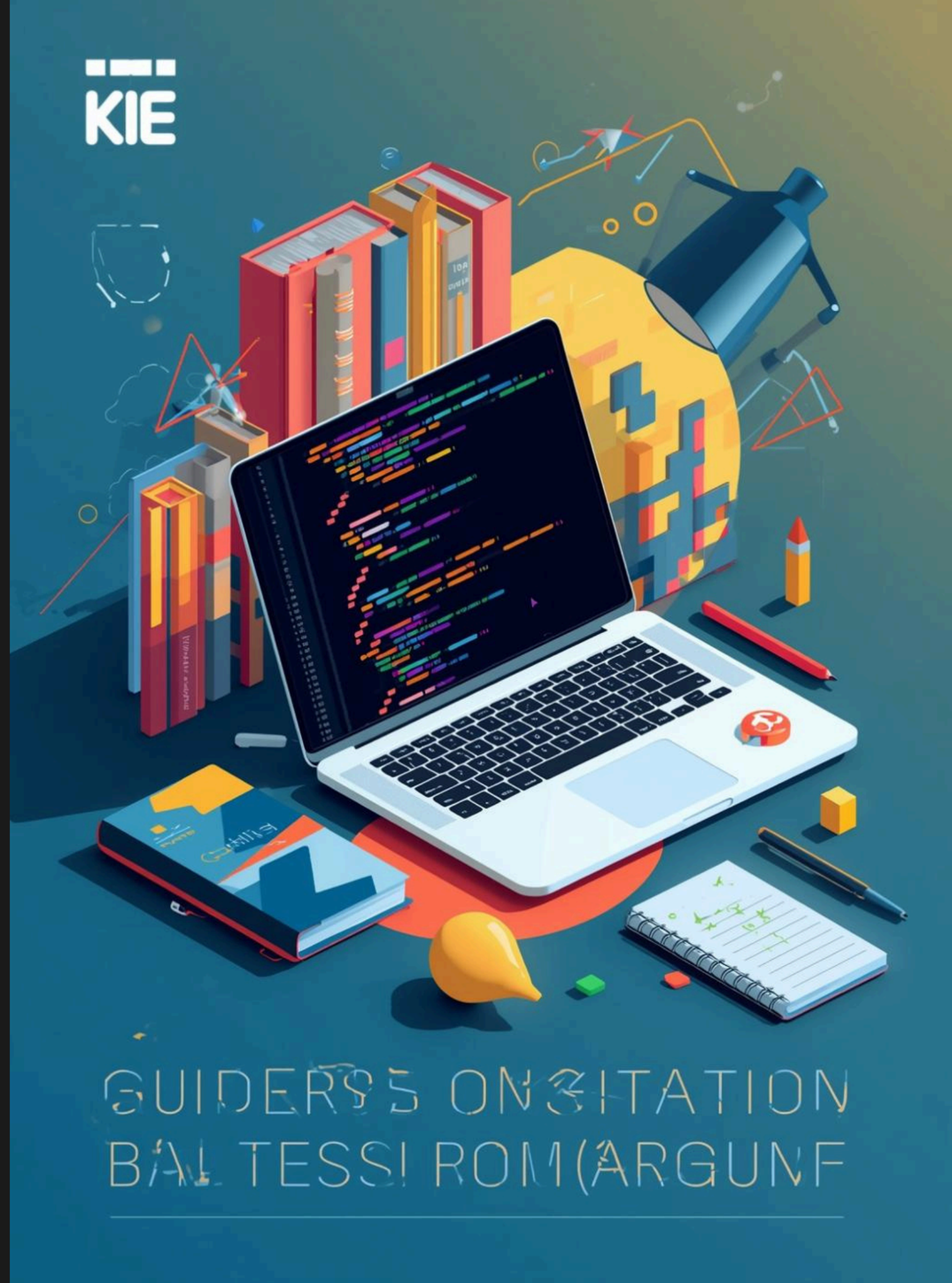


Persistencia de Datos

La **persistencia de datos** es esencial para mantener un registro de las puntuaciones del juego, garantizando que la información se conserve más allá de la ejecución del programa.

```python
def obtener_max_puntaje():
    """Función para leer el récord desde un archivo."""
    if not os.path.exists("record.txt"):
        with open("record.txt", "w") as f:
            f.write("0")
        return 0
    with open("record.txt", "r") as f:
        return int(f.read())

def guardar_max_puntaje(nuevo_record):
    """Función para escribir el récord en el archivo."""
    with open("record.txt", "w") as f:
        f.write(str(nuevo_record))

# Ejemplo de Programación Funcional: Lambdas para validaciones puras
esta_fuera_limites = lambda pos, limite: abs(pos) > limite
calcular_distancia = lambda pos1, pos2: pos1.distance(pos2)
```



Conclusiones

En conclusión, el proyecto "Snake Game" ha sido un **éxito total**, demostrando la aplicación práctica de los conceptos aprendidos en LÓGICA DE PROGRAMACIÓN